JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util.stream

Interface BaseStream<T,S extends BaseStream<T,S>>

Type Parameters:

`T - the type of the stream elements`

`S - the type of the stream implementing BaseStream`

All Superinterfaces:

`AutoCloseable`

All Known Subinterfaces:

`DoubleStream, IntStream, LongStream, Stream<T>`

```
public interface BaseStream<T,S extends BaseStream<T,S>>
extends AutoCloseable
```

Base interface for streams, which are sequences of elements supporting sequential and parallel aggregate operations. The following example illustrates an aggregate operation using the stream types `Stream` and `IntStream`, computing the sum of the weights of the red widgets:

```
int sum = widgets.stream()
                 .filter(w -> w.getColor() == RED)
                 .mapToInt(w -> w.getWeight())
                 .sum();
```

See the class documentation for `Stream` and the package documentation for java.util.stream for additional specification of streams, stream operations, stream pipelines, and parallelism, which governs the behavior of all stream types.

Since:

1.8

See Also:

Stream, IntStream, LongStream, DoubleStream, java.util.stream

Method Summary

Abstract Methods

Modifier and Type	Method	Description
void	close()	Closes this stream, causing all close handlers for this stream pipeline to be called.
boolean	isParallel()	Returns whether this stream, if a terminal operation were to be executed, would execute in parallel.
Iterator<T>	iterator()	Returns an iterator for the elements of this stream.
S	onClose(Runnable closeHandler)	Returns an equivalent stream with an additional close handler.
S	parallel()	Returns an equivalent stream that is parallel.
S	sequential()	Returns an equivalent stream that is sequential.
Spliterator<T>	spliterator()	Returns a spliterator for the elements of this stream.
S	unordered()	Returns an equivalent stream that is **unordered**.

Method Detail

iterator

`Iterator<T> iterator()`

Returns an iterator for the elements of this stream.

This is a terminal operation.

Returns:

the element iterator for this stream

spliterator

`Spliterator<T> spliterator()`

Returns a spliterator for the elements of this stream.

This is a terminal operation.

Returns:

the element spliterator for this stream

isParallel

`boolean isParallel()`

Returns whether this stream, if a terminal operation were to be executed, would execute in parallel. Calling this method after invoking an terminal stream operation method may yield unpredictable results.

Returns:

true if this stream would execute in parallel if executed

sequential

```
S sequential()
```

Returns an equivalent stream that is sequential. May return itself, either because the stream was already sequential, or because the underlying stream state was modified to be sequential.

This is an intermediate operation.

Returns:

a sequential stream

parallel

```
S parallel()
```

Returns an equivalent stream that is parallel. May return itself, either because the stream was already parallel, or because the underlying stream state was modified to be parallel.

This is an intermediate operation.

Returns:

a parallel stream

unordered

```
S unordered()
```

Returns an equivalent stream that is unordered. May return itself, either because the stream was already unordered, or because the underlying stream state was modified to be unordered.

This is an intermediate operation.

Returns:

an unordered stream

onClose

```
S onClose(Runnable closeHandler)
```

Returns an equivalent stream with an additional close handler. Close handlers are run when the `close()` method is called on the stream, and are executed in the order they were added. All close handlers are run, even if

earlier close handlers throw exceptions. If any close handler throws an exception, the first exception thrown will be relayed to the caller of `close()`, with any remaining exceptions added to that exception as suppressed exceptions (unless one of the remaining exceptions is the same exception as the first exception, since an exception cannot suppress itself.) May return itself.

This is an intermediate operation.

Parameters:

`closeHandler - A task to execute when the stream is closed`

Returns:

`a stream with a handler that is run if the stream is closed`

close

`void close()`

Closes this stream, causing all close handlers for this stream pipeline to be called.

Specified by:

`close` in interface `AutoCloseable`

See Also:

`AutoCloseable.close()`

Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms. Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.